Práctica 5. Datos Anómalos

Análisis Exploratorio de Datos, Máster en Ciencia de Datos - UV

Contents

Introducción y objetivos **1**

Detección de inconsistencias en los datos **2**

Ejercicio 1: Inspección Visual 2

Ejercicio 2: Valores especiales 5

Ejercicio 3: Valores perdidos y observaciones completas 5

Detección de outliers univariante **6**

Ejercicio 4: Estadísticos básicos 6

Ejercicio 5: Métodos de detección de outliers 7

Ejercicio 6: Detección de outliers 8

Ejercicio 7: Comparativa de los métodos de detección de outliers 8

Ejercicio 8: Representación gráfica 9

Imputación de datos **10**

Ejercicio 9: Imputación de datos perdidos con estadísticos básicos 11

Ejercicio 10: Representación gráfica 11


```
library(pacman)
packages = c("knitr", "ggplot2","tidyr","dplyr","readr")
pacman::p_load(char=packages)
```

Introducción y objetivos

El objetivo de esta práctica es comprobar si los conjuntos de datos con los que vamos a trabajar son consistentes y, por tanto, pueden ser empleados para realizar un análisis sin errores. Las observaciones contenidas en los datos pueden presentar diferentes tipos de inconsistencias, como valores perdidos (missing values), valores especiales y valores anómalos (outliers).

Para evitar estas inconsistencias, estos datos deberán ser eliminados, corregidos o imputados. Siendo las etapas que pueden seguirse para obtener datos consistentes:

1. Detección de las inconsistencias.
2. Detectar el campo o campos que la generan y las dependencias entre ellos.

3. Corrección de errores.

Vamos a trabajar con el conjunto de datos *dirty_iris*, que ya conocemos, que contiene datos perdidos, valores especiales y anomalías de carácter numérico respecto a la distribución de los datos (`dirty_iris.csv`).

```
rm(list=ls())
dirty_iris <- read_csv("data/dirty_iris.csv")
```

Detección de inconsistencias en los datos

Por lo general, no hay un convenio o una representación común para indicar un dato perdido o incluso determinados valores especiales. En los siguientes ejercicios vamos a analizar cómo detectar inconsistencias presentes en los datos.

Ejercicio 1: Inspección Visual

Inspecciona visualmente los valores del conjunto de datos (`View`) y ordena de mayor a menor y de menor a mayor cada variable para descubrir las diferentes inconsistencias que presenta el conjunto `dirty_iris`. Comenta los valores que te llamen la atención.

```
# Medidas con valores faltantes, longitudes negativas y cero

#View(dirty_iris)
dirty_iris %>% arrange(Sepal.Length)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1          0          NA            1.3         0.4 setosa
 2          4.3         3            1.1         0.1 setosa
 3          4.4         3.2         NA           0.2 setosa
 4          4.4         2.9          1.4         0.2 setosa
 5          4.4         3            1.3        NA   setosa
 6          4.5         2.3          1.3         0.3 setosa
 7          4.6         3.2          1.4         0.2 setosa
 8          4.6         3.4          1.4         0.3 setosa
 9          4.6         3.1          1.5         0.2 setosa
10          4.6         3.6         NA           0.2 setosa
# i 140 more rows
```

```
dirty_iris %>% arrange(Sepal.Width)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1          5          -3            3.5         1   versicolor
 2          5.7         0            1.7         0.3 setosa
 3          6.2         2.2          4.5         1.5 versicolor
 4          6           2.2          5           1.5 virginica
 5          6           2.2          4           1   versicolor
```

```
 6          5           2.3          3.3        NA   versicolor
 7          5.5         2.3          4           1.3 versicolor
 8          4.5         2.3          1.3         0.3 setosa
 9          6.7         2.5          5.8         1.8 virginica
10          6.3         2.5          4.9         1.5 versicolor
# i 140 more rows
```

```
dirty_iris %>% arrange(Petal.Length)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1          5.1         3.8        0             0.2 setosa
 2         NA           2.8        0.82          1.3 versicolor
 3          5.5        NA          0.925         1   versicolor
 4          4.3         3          1.1           0.1 setosa
 5          5          NA          1.2           0.2 setosa
 6          4.7         3.2        1.3           0.2 setosa
 7          0          NA          1.3           0.4 setosa
 8          5           3.5        1.3           0.3 setosa
 9          5.5         3.5        1.3           0.2 setosa
10          4.5         2.3        1.3           0.3 setosa
# i 140 more rows
```

```
dirty_iris %>% arrange(Petal.Width)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1          4.9         3.6         NA           0.1 setosa
 2          4.9         3.1          1.5         0.1 setosa
 3          4.8         3            1.4         0.1 setosa
 4          4.3         3            1.1         0.1 setosa
 5          5.2         4.1          1.5         0.1 setosa
 6          5.3        NA           NA           0.2 setosa
 7          4.8         3.1          1.6         0.2 setosa
 8          4.7         3.2          1.3         0.2 setosa
 9         NA           4           NA           0.2 setosa
10          4.6         3.2          1.4         0.2 setosa
# i 140 more rows
```

```
dirty_iris %>% arrange(-Sepal.Length)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1         73          29           63          NA   virginica
 2         49          30           14           2   setosa
 3          7.9         3.8          6.4         2   virginica
 4          7.7         2.8          6.7         2   virginica
 5          7.7         3.8          6.7         2.2 virginica
 6          7.7         2.6          6.9         2.3 virginica
```

```
 7          7.7         3           NA           2.3 virginica
 8          7.6         3            6.6         2.1 virginica
 9          7.4         2.8          6.1         1.9 virginica
10          7.2         3            5.8         1.6 virginica
# i 140 more rows
```

```
dirty_iris %>% arrange(-Sepal.Width)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1         49          30           14           2   setosa
 2         73          29           63          NA   virginica
 3          5.5         4.2          1.4         0.2 setosa
 4          5.2         4.1          1.5         0.1 setosa
 5         NA           4           NA           0.2 setosa
 6         NA           3.9          1.7         0.4 setosa
 7          7.7         3.8          6.7         2.2 virginica
 8          5.1         3.8          1.5         0.3 setosa
 9          5.1         3.8          0           0.2 setosa
10          7.9         3.8          6.4         2   virginica
# i 140 more rows
```

```
dirty_iris %>% arrange(-Petal.Length)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1         73          29           63          NA   virginica
 2          6.6         2.9         23           1.3 versicolor
 3         49          30           14           2   setosa
 4          7.7         2.6          6.9         2.3 virginica
 5          7.7         2.8          6.7         2   virginica
 6          7.7         3.8          6.7         2.2 virginica
 7          7.6         3            6.6         2.1 virginica
 8          7.9         3.8          6.4         2   virginica
 9          7.4         2.8          6.1         1.9 virginica
10          6.3         3.3          6           2.5 virginica
# i 140 more rows
```

```
dirty_iris %>% arrange(-Petal.Width)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1          5.1         3.8          1.9       Inf   setosa
 2          6.3         3.3          6           2.5 virginica
 3          6.7         3.3          5.7         2.5 virginica
 4          7.2         3.6         NA           2.5 virginica
 5          6.3         3.4         NA           2.4 virginica
 6          5.8        NA            5.1         2.4 virginica
 7          6.7         3.1          5.6         2.4 virginica
```

```
 8          6.2        NA           5.4         2.3 virginica
 9         NA           3.2         5.7         2.3 virginica
10          6.9         3.1         5.1         2.3 virginica
# i 140 more rows
```

Podemos ver como hay valores extremales que no tienen ninngún sentido (sepalos 10 veces mas largos que los demas o incluso un sepalo con anchura negativa). También hay valores de 0 que tampoco tienen sentido, a parte de los múltiples valores Na que hay en el dataset.

Ejercicio 2: Valores especiales

Visualizar los datos puede ser muy revelador, pero en general, especialmente para conjuntos de datos grandes, realizaremos este análisis de forma automática.

Los valores especiales más comunes en R son:

- NA (*not available*): missing value o dato perdido (`is.na`)
- NULL (*empty set*): elemento nulo de longitud 0 (`is.null`)
- NaN (*not a number*): indeterminación con resultado desconocido (`is.nan`)
- Inf (*infinite*): valor numérico de valor infinito (`is.infinite`)

Nota: A parte de estas funciones, `is.numeric` comprueba si la variable es numérica; `is.finite` comprueba la existencia de cualquier valor especial en un vector numérico pero devuelve `FALSE` para variables de texto.

Reemplaza, en todas las columnas numéricas del data frame `dirty_iris`, los valores que sean especiales (usando `is.finite`), negativos, o iguales a cero, por `NA`. Nota: Puedes reemplazar los valores para cada variable del dataframe usando `mutate` con la función `ifelse`.

```
# DEBEMOS ELIMINAR TAMBIEN LOS VALROES IGUALES A CERO YA QUE SE TRATA DE LONGITUDES DE PARTES DE UNA FL

dirty_iris_clean <- dirty_iris %>%
  mutate(across(where(is.numeric), ~ ifelse(!is.finite(.) | . <= 0, NA, .)))
```

Con este comando cambiamos las columnas numéricas, y sustituimos solo ls valores que son menores o iguales a 0 y los otrros valores especiales por Na.

Ejercicio 3: Valores perdidos y observaciones completas

- Calcula el número y porcentaje de observaciones completas (muestras sin ningún dato perdido) empleando `complete.cases` (detecta todos los registros sin datos faltantes).
- Emplea después la función `na.omit` para quedarnos solo con las filas con datos completos en un nuevo data frame `clean_iris`.

```
filas_completas <- complete.cases(dirty_iris_clean)

ncompletas <- sum(filas_completas)

ncompletas
```

```
[1] 92
```

```r
nrow(dirty_iris)
```

```
[1] 150
```

```r
clean_iris <- na.omit(dirty_iris_clean)
```

Comprobamos que el total de filas sin Na son unas 92 de 150. Utilizando na.omit podemos quedarnos con esas 92 instancias.

Detección de outliers univariante

Un outlier es una observación (o conjunto de observaciones) que no parecen consistentes con el resto de valores de esa variable. Por lo general, esas observaciones divergirán del patrón de comportamiento del resto de datos, es decir, presentarán una desviación respecto de la distribución de los datos.

En esta sección, vamos a centrarnos en métodos de detección univariantes, así que, en los ejercicios, a modo de ejemplo, analizaremos solo la variable `Petal.Length`.

Ejercicio 4: Estadísticos básicos

La presencia de outliers puede producir una gran influencia en caracterizaciones sencillas de los datos, como la media y la desviación típica.

Empleado la función `summarise y across`, calcula la media, la desviación típica, mediana, la *median absolute deviation from the median* (MADM) y el *interquartile range* (IQR) de las variables numéricas. Hazlo para el data frame con observaciones completas `clean_iris` (que no tiene NA pero tiene menos observaciones) y para el data frame con datos faltantes `dirty_iris` (que presenta muchos NA pero tiene todas las observaciones).

- Ten en cuenta que no se puede realizar un análisis estadístico con datos faltantes, pero en la mayoría de los casos se puede optar por no considerar dichos valores al realizar los cálculos (parámetro `na.rm=TRUE`).
- Para obtener el valor MADM, escribe una función `madm(x)` que sea robusta a la presencia de valores NA. Recuerda que el MADM se calcula como $1.4826 \cdot \text{median}(|x - \text{median}(\text{x})|)$.
- Compara los resultados para los dos conjuntos en la variable `Petal.Length`.

```r
library(dplyr)


resumen_clean <- clean_iris %>%
  summarise(across(where(is.numeric), list(
    mean   = ~mean(.),
    sd     = ~sd(.),
    median = ~median(.),
    madm   = ~mad(.),
    iqr    = ~IQR(.)
  ), .names = "{.col}_{.fn}"))


resumen_dirty <- dirty_iris %>%
  summarise(across(where(is.numeric), list(
    mean   = ~mean(., na.rm = TRUE),
```

```
    sd     = ~sd(., na.rm = TRUE),
    median = ~median(., na.rm = TRUE),
    madm   = ~mad(., na.rm = TRUE),
    iqr    = ~IQR(., na.rm = TRUE)
  ), .names = "{.col}_{.fn}"))

resumen_clean
```

```
# A tibble: 1 x 20
  Sepal.Length_mean Sepal.Length_sd Sepal.Length_median Sepal.Length_madm Sepal.Length_iqr
              <dbl>           <dbl>                <dbl>             <dbl>            <dbl>
1              6.37            4.57                  5.8              1.04              1.3
# i 15 more variables: Sepal.Width_mean <dbl>, Sepal.Width_sd <dbl>, Sepal.Width_median <dbl>,
#   Sepal.Width_madm <dbl>, Sepal.Width_iqr <dbl>, Petal.Length_mean <dbl>, Petal.Length_sd <dbl>,
#   Petal.Length_median <dbl>, Petal.Length_madm <dbl>, Petal.Length_iqr <dbl>,
#   Petal.Width_mean <dbl>, Petal.Width_sd <dbl>, Petal.Width_median <dbl>, Petal.Width_madm <dbl>,
#   Petal.Width_iqr <dbl>
```

```
resumen_dirty
```

```
# A tibble: 1 x 20
  Sepal.Length_mean Sepal.Length_sd Sepal.Length_median Sepal.Length_madm Sepal.Length_iqr
              <dbl>           <dbl>                <dbl>             <dbl>            <dbl>
1              6.56            6.80                 5.75             0.964              1.3
# i 15 more variables: Sepal.Width_mean <dbl>, Sepal.Width_sd <dbl>, Sepal.Width_median <dbl>,
#   Sepal.Width_madm <dbl>, Sepal.Width_iqr <dbl>, Petal.Length_mean <dbl>, Petal.Length_sd <dbl>,
#   Petal.Length_median <dbl>, Petal.Length_madm <dbl>, Petal.Length_iqr <dbl>,
#   Petal.Width_mean <dbl>, Petal.Width_sd <dbl>, Petal.Width_median <dbl>, Petal.Width_madm <dbl>,
#   Petal.Width_iqr <dbl>
```

Es complicado tener en cuenta todas las métricas de cada columna, pero comparando los datos limpiados con los de sin limpiar podemos decir que no hay mucho cambio en medianas, madm rangos intercuartilicos, en medias si que hay algo más pero lo que más destaca es que en los datos limpios la desviacion típica sí baja considerablemente, ya que hemos eliminado datos que podían estar bastante lejos de la distribución.

Ejercicio 5: Métodos de detección de outliers

Escribe una función que detecte los outliers de un vector x para cada uno de los siguientes métodos de detección de outliers univariantes (todas las funciones tienen que ser robustas a la presencia de valores NA).

- **Regla 3 sigma** `reglasigma(x)`: Considera que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la media y la dispersión por la varianza. Valores por encima de 3 desviaciones típicas se consideran outliers.
- **Identificador Hampel** `reglahampel(x)`: Utiliza estimadores robustos y no presupone que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la mediana y la dispersión por la *median absolute deviation from the median* (MADM). Valores por encima de 3 MADM se consideran outliers.
- **Regla boxplot** `reglaboxplot(x)`: A partir de los datos se calcula su diagrama de caja (boxplot) y se definen como datos típicos superiores hasta el cuartil 75%, x_s; datos típicos inferiores hasta el cuartil 25%, x_i; y la dispersión se define como el rango intercuartílico ($IQR = x_s - x_i$). Valores por encima de $x_s + 1.5 \cdot IQR$ o por debajo de $x_i - 1.5 \cdot IQR$ se consideran outliers. Usa la función `quantile` para calcular los cuartiles.

- **Percentiles** `reglapercentil(x)`: Cualquier valor que esté fuera del rango del percentil 5% y 95% se puede considerar como atípico. Usa la función `quantile` para calcular los percentiles.

Hacemos las 4 funciones para que nos devuelvan un vector lógico del mismo tamaño de x para el proximo apartado. Ademas, he tenido que cambiar para que solo tengan un argumento y sea un vector

Ejercicio 6: Detección de outliers

Utiliza la función `mutate` y `across` para identificar los outliers en la variable `Petal.Length` con todos los métodos de detección de outliers implementados. Genera una columna nueva para cada método. Nota: podría extenderse la aplicación a todas las variables numéricas usando también `mutate`, `across` y `where`.

```
outliers_petalL <- dirty_iris %>%
  mutate(across(
    .cols = Petal.Length,
    .fns = list(
      sigma     = reglasigma,
      hampel    = reglahampel,
      boxplot   = reglaboxplot,
      percentil = reglapercentil
    ),
    .names = '.{fn}')
  )
```

Tras hacer cambios en las funciones, simplemente utilizamos el mutate para hacer las 4 columnas que nos dicen si el valor es outlier para cada metodo (para la columa petal.lenght)

Ejercicio 7: Comparativa de los métodos de detección de outliers

En los métodos de detección de outliers analizados, examina los resultados comparando:

- Número de outliers detectados por método.
- Los valores etiquetados como outliers (ordenados de menor a mayor).

```
n<-c('n_sigma'=sum(outliers_petalL$.sigma),'n_hampel'=sum(outliers_petalL$.hampel), 'n_boxplot'=sum(outl

n
```

```
    n_sigma    n_hampel   n_boxplot n_percentil
          2           3           3          12
```

```
outliers_sigma<-dirty_iris[outliers_petalL$.sigma,]
outliers_sigma
```

```
# A tibble: 2 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
         <dbl>       <dbl>        <dbl>       <dbl> <chr>
1         73          29             63        NA   virginica
2          6.6         2.9           23         1.3 versicolor
```

```
outliers_hampel<-dirty_iris[outliers_petalL$.hampel,]
outliers_hampel
```

```
# A tibble: 3 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
         <dbl>       <dbl>        <dbl>       <dbl> <chr>
1         73          29             63        NA   virginica
2          6.6         2.9           23         1.3 versicolor
3         49          30             14         2   setosa
```

```
outliers_boxplot<-dirty_iris[outliers_petalL$.boxplot,]
outliers_boxplot
```

```
# A tibble: 3 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
         <dbl>       <dbl>        <dbl>       <dbl> <chr>
1         73          29             63        NA   virginica
2          6.6         2.9           23         1.3 versicolor
3         49          30             14         2   setosa
```

```
outliers_percentil<-dirty_iris[outliers_petalL$.percentil,]
outliers_percentil
```

```
# A tibble: 12 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1         73          29          63            NA   virginica
 2         NA           2.8         0.82          1.3 versicolor
 3          6.6         2.9         23            1.3 versicolor
 4          5          NA           1.2           0.2 setosa
 5          7.7         2.8         6.7           2   virginica
 6          7.7         3.8         6.7           2.2 virginica
 7          4.3         3           1.1           0.1 setosa
 8          7.6         3           6.6           2.1 virginica
 9          5.1         3.8         0             0.2 setosa
10          5.5        NA           0.925         1   versicolor
11          7.7         2.6         6.9           2.3 virginica
12         49          30          14             2   setosa
```

Podemos ver como de laxos son estos métodos. El de sigma es el que más laxo es, marcando solo 2 outliers, que tambien estan contenidos en los outliers marcados por el boxplot y por hampel. Estos dos ultimos marcan 3 outliers, mientras que el ultimo metodo es el mas radical y marca 12 outliers.

Ejercicio 8: Representación gráfica

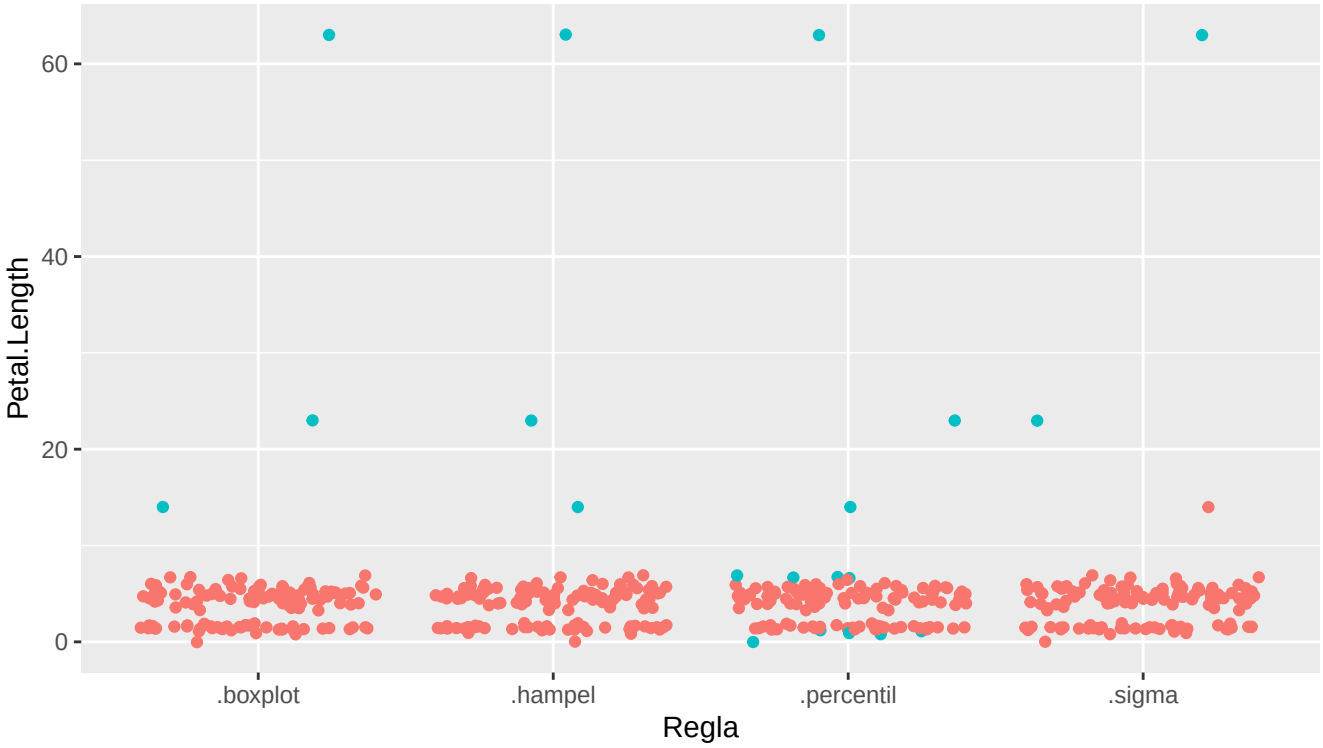
Representa gráficamente los datos de `Petal.Length` (distribución de puntos en el eje y) para todas las reglas de detección implementadas (repartidas en el eje x) mostrando los valores identificados como típicos y los identificados como outliers en diferente color. Usa la función `pivot_longer` para poder agrupar por reglas y colorear los outliers fácilmente.

Nota: usa en `geom_point` con la opción `position='jitter'` para mejorar la legibilidad del gráfico.

```r
library(ggplot2)
library(tidyr)


outliers_long <- outliers_petalL %>%
  pivot_longer(
    cols = c(.sigma, .hampel, .boxplot, .percentil),
    names_to = "regla",
    values_to = "es_outlier"
  )

ggplot(outliers_long, aes(x = regla, y = Petal.Length, color = es_outlier)) +
  geom_point(position = "jitter") +
  labs(x = "Regla", y = "Petal.Length", color = "Outlier")
```



Primero hacemos los ddatos tidy, dividiendo en dos columnas, para asi poder hacer los boxplots con el color cambiado con la variable es_outlier. De esta manera, tenemos los 4 boxplots donde se ve claramente que los puntos que mas se separan se clasificn como outliers, aunque como hemos visto antes algunos metodos son mas permisivos.

Imputación de datos

Como se ha visto, con *missing data* nos referimos a una observación (o conjunto de observaciones) cuyo valor no es conocido.

Vamos a ver a continuación ejemplos de cómo imputar valores realistas a esas observaciones perdidas.

Ejercicio 9: Imputación de datos perdidos con estadísticos básicos

La opción más sencilla es imputar con un valor estimado de los valores conocidos (por ejemplo, con la media, la mediana, o la moda).

Se puede realizar una imputación generalizada (usando todos los valores de la variable) o una imputación a partir de casos similares (la estimación se realiza teniendo en cuenta otra variable):

- Imputa los missing data en `Petal.Length` empleando la media. Nota: Puedes reemplazar los valores vacios usando `mutate` con `ifelse` creando una nueva variable `Petal.Length1`.

```
dirty_iris <- dirty_iris %>%
  mutate(Petal.Length1 = ifelse(
    is.na(Petal.Length),
    mean(Petal.Length, na.rm = TRUE),
    Petal.Length
  ))
```

Aqui simplemente empleamos mutate para que con in ifelse cambiar los na por la media de longitud del petalo con na.rm=TRUE.

- Imputa los missing data en `Petal.Length` empleando en esta ocasión la media para cada tipo de `Species` (usar`group_by` y `mutate`) creando una nueva variable `Petal.Length2`.

```
dirty_iris <- dirty_iris %>%
  group_by(Species) %>%
  mutate(Petal.Length2 = ifelse(
    is.na(Petal.Length),
    mean(Petal.Length, na.rm = TRUE),
    Petal.Length
  ))
```

Aqui, primero agrupamos por species y luego hacemos exactamente lo mismo.

Ejercicio 10: Representación gráfica

Vamos a comparar ahora los dos métodos de imputación gráficamente eliminando los outliers para una mejor visualización.

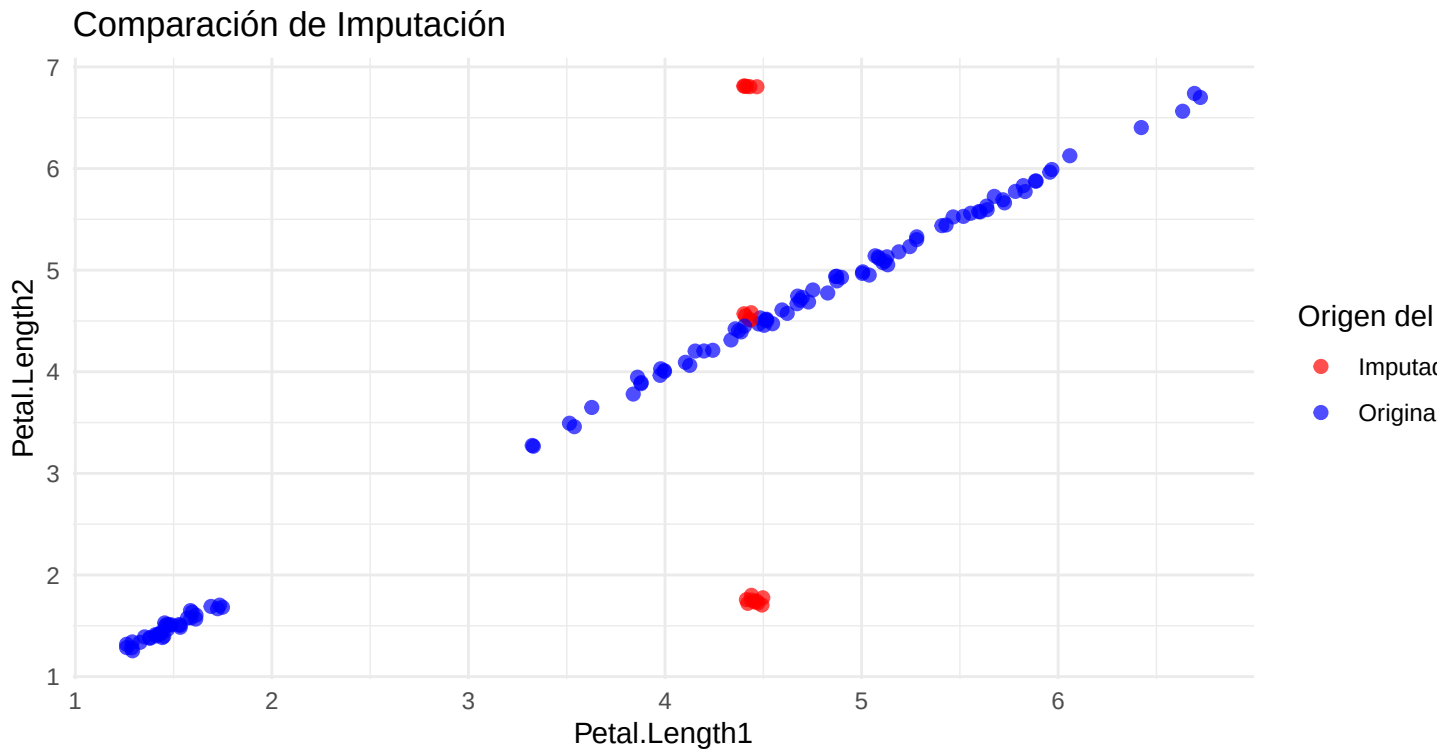
- Recalcula las reglas de identificación de outliers (Ejercicio 6) para la nueva variable `Petal.Length2`.

```
dirty_iris <- dirty_iris %>%
  mutate(out_percentil_PL2 = reglapercentil(Petal.Length2))
```

- Representa los dos tipos de imputación empleados en un gráfico de dispersión (`Petal.Length1` vs. `Petal.Length2`) eliminando los outliers identificados con la regla `reglapercentil` y poniendo en diferente color los datos originales y los imputados.

```r
data_plot <- dirty_iris %>%
  filter(!out_percentil_PL2) %>%
  mutate(
    tipo_dato = ifelse(is.na(Petal.Length), "Imputado", "Original")
  )


ggplot(data_plot, aes(x = Petal.Length1, y = Petal.Length2, color = tipo_dato)) +
  geom_point(position = position_jitter(width = 0.05, height = 0.05), alpha = 0.7, size = 2) +
  scale_color_manual(values = c("Imputado" = "red", "Original" = "blue")) +
  labs(
    title = "Comparación de Imputación",
    x = "Petal.Length1",
    y = "Petal.Length2",
    color = "Origen del Dato"
  ) +
  theme_minimal()
```



Tras recalcular la regla, podemos ver en la gráfica como parece ser que al imputar por grupos probablemnete el error en general sea menor que si imputamos por la media en general.